

Program 1: Perform push and pop operations in stack

Source Code

```
#include <iostream>
using namespace std;

class myStack {
    int max_size;
    int* arr;
    int top = -1;
public:
    myStack(int _max_size = 10){
        max_size = _max_size;
        arr = new int[max_size];
    }
    bool push(int num_to_push){
        if (top == max_size - 1){
            std::cout << "Stack Overflow!!" << std::endl;
            return false;
        }
        top += 1;
        arr[top] = num_to_push;
        return true;
    }
    int pop(){
        if (top == -1){
            std::cout << "Stack Underflow!!" << std::endl;
            return 0;
        }
        top -= 1;
        return arr[top+1];
    }
    int peek(){
        if (top == -1){
            std::cout << "Stack Empty!!" << std::endl;
            return 0;
        }
        return arr[top];
    }
};

int main(){
    myStack stac = myStack(15);
    bool check = stac.push(100);
    std::cout << "Popped: " << stac.pop() << std::endl;
    int test = stac.pop();
}
```

Output

```
pawanadhikari@Pawans-MacBook-Air 01_Stack % ./01_Stack  
Popped: 100  
Stack Underflow!!
```

Program 2.1: Infix Expression to Postfix

Source Code

```
#include <iostream>
using namespace std;
#include <stack>
#include <string>
#include <map>

class Infix {
public:
    string toScan;
    stack<char> myStack;
    Infix(string toScan_){
        toScan = toScan_;
    }
    string toPostfix(){
        string Tape = toScan;
        string outString = "";
        std::map<char, int> PrecedenceAssocMap = {
            {'+', 1},
            {'-', 1},
            {'*', 2},
            {'/', 2}
        };
        while (!Tape.empty()){
            if (!myStack.empty()) cout << "Current Stack Top = " <<
myStack.top() <<endl;
            char cchar = char(Tape[0]);
            Tape = Tape.substr(1);
            if ((cchar >= 'A' && cchar <= 'Z') || (cchar >= 'a' && cchar <=
'z')){
                outString += cchar;
            }
            else if (cchar == '('){
                myStack.push(cchar);
            }
            else if (cchar == ')'){
                char dchar;
                if (!myStack.empty()) {
                    dchar = myStack.top();
                    myStack.pop();
                }
                while (dchar != '('){
                    outString+= dchar;
                    if (!myStack.empty()){
                        dchar = myStack.top();
                        myStack.pop();
                    }
                }
            }
        }
    }
};
```

```

    }
    else {
        while (!myStack.empty() && myStack.top() != '(' &&
PrecedenceAssocMap.at(cchar) <= PrecedenceAssocMap.at(myStack.top())){
            if (!myStack.empty()) {
                outString += myStack.top();
                myStack.pop();
            }
            myStack.push(cchar);
        }
    }
    while (!myStack.empty()){
        char current_char = myStack.top();
        myStack.pop();
        outString += current_char;
    }
    return outString;
}
};

int main(){
    cout << "Input you required infix string." << endl;
    string inpString;
    cin >> inpString;
    Infix inf = Infix(inpString);
    cout << "The input infix string was: " << inf.toScan << endl;
    string outString = inf.toPostfix();
    cout << "The infix string in postfix is : " << outString << endl;
}

```

Output

```

pawanadhikari@Pawans-MacBook-Air 01_Stack % ./02.1_Postfix
Input you required infix string.
a+b-c*d
The input infix string was: a+b-c*d
Current Stack Top = +
Current Stack Top = +
Current Stack Top = -
Current Stack Top = -
Current Stack Top = *
The infix string in postfix is : ab+cd*-

```

Program 1.2: Infix Expression to Prefix

Source Code

```
#include <iostream>
using namespace std;
#include <stack>
#include <string>
#include <map>

class Infix {
public:
    string toScan;
    stack<char> myStack;
    Infix(string toScan_){
        toScan = toScan_;
    }
    string toPrefix(){
        string Tape = toScan;

        reverse(Tape.begin(), Tape.end());
        for (char &ch : Tape) {
            if (ch == '(') ch = ')';
            else if (ch == ')') ch = '(';
        }

        string outString = "";
        std::map<char, int> PrecedenceAssocMap = {
            {'+', 1},
            {'-', 1},
            {'*', 2},
            {'/', 2}
        };
        while (!Tape.empty()){
            if (!myStack.empty()) cout << "Current Stack Top = " <<
myStack.top() <<endl;
            char cchar = char(Tape[0]);
            Tape = Tape.substr(1);
            if ((cchar >= 'A' && cchar <= 'Z') || (cchar >= 'a' && cchar <=
'z')){
                outString += cchar;
            }
            else if (cchar == '('){
                myStack.push(cchar);
            }
            else if (cchar == ')'){
                char dchar;
                if (!myStack.empty()) {
                    dchar = myStack.top();
                    myStack.pop();
                }
            }
        }
    }
};
```

```

        while (dchar != '('){
            outString+= dchar;
            if (!myStack.empty()){
                dchar = myStack.top();
                myStack.pop();
            }
        }
    }
    else {
        while (!myStack.empty() && myStack.top()!='(' &&
PrecedenceAssocMap.at(cchar) < PrecedenceAssocMap.at(myStack.top())){
            if (!myStack.empty()) {
                outString += myStack.top();
                myStack.pop();
            }
        }
        myStack.push(cchar);
    }
}
while (!myStack.empty()){
    char current_char = myStack.top();
    myStack.pop();
    outString += current_char;
}
reverse(outString.begin(), outString.end());
return outString;
}
};

int main(){
    cout << "Input you required infix string." << endl;
    string inpString;
    cin >> inpString;
    Infix inf = Infix(inpString);
    cout << "The input infix string was: " << inf.toScan << endl;
    string outString = inf.toPrefix();
    cout << "The infix string in prefix is : " << outString << endl;
}

```

Output

```

pawanadhikari@Pawans-MacBook-Air 01_Stack % ./02.2_Prefix
Input you required infix string.
a+b-c*d
The input infix string was: a+b-c*d
Current Stack Top = *
Current Stack Top = *
Current Stack Top = -
Current Stack Top = -

```

Current Stack Top = +
The infix string **in** prefix is : -+ab*cd

Program 3: Evaluate the postfix expression

Source Code

```
#include <iostream>
using namespace std;
#include <stack>
#include <string>
#include <map>

class PostfixExp {
public:
    string toScan;
    stack<int> myStack;
    PostfixExp(string toScan_){
        toScan = toScan_;
    }
    int operate(int a, int b, char op){
        if (op == '+'){
            return a+b;
        }
        else if (op == '-'){
            return a-b;
        }
        else if (op == '*'){
            return a*b;
        }
        else if (op == '/'){
            return a/b;
        }
        else {
            cout << "Unsupported Operand";
            return 1;
        }
    }
    int evaluate(){
        string Tape = toScan;

        string outString = "";
        std::map<char, int> PrecedenceAssocMap = {
            {'+', 1},
            {'-', 1},
            {'*', 2},
            {'/', 2}
        };
        while (!Tape.empty()){
            if (!myStack.empty()) cout << "Current Stack Top = " <<
myStack.top() <<endl;
            char cchar = char(Tape[0]);
            int num = cchar - '0';
            Tape = Tape.substr(1);
```



```

        if ((cchar >= '0' && cchar <= '9')){
            myStack.push(num);
        }
        else if(PrecedenceAssocMap.at(cchar)) {
            int op1, op2;
            if (!myStack.empty()){
                op2 = myStack.top();
                myStack.pop();
            }
            else cout << "Invalid!" <<endl;
            if (!myStack.empty()){
                op1 = myStack.top();
                myStack.pop();
            }
            else cout << "Invalid!" <<endl;
            int result = this->operate(op1, op2, cchar);
            myStack.push(result);
        }
    }
    return myStack.top();
}
};

int main(){
    cout << "Input your required postfix expression." << endl;
    string inpString;
    cin >> inpString;
    PostfixExp inf = PostfixExp(inpString);
    cout << "The input postfix expression was: " << inf.toScan << endl;
    int result = inf.evaluate();
    cout << inpString << "=" << result << endl;
}

```

Output

```

pawanadhikari@Pawans-MacBook-Air 01_Stack % ./03_Evaluate
Input your required postfix expression.
12+34*-
The input postfix expression was: 12+34*-
Current Stack Top = 1
Current Stack Top = 2
Current Stack Top = 3
Current Stack Top = 3
Current Stack Top = 4
Current Stack Top = 12
12+34*-=-9

```

Program 4: Check the paired parenthesis in mathematical expression

Source Code

```
#include <iostream>
using namespace std;
#include <stack>
#include <string>
#include <map>

class PairedParanthesis {
public:
    string toScan;
    stack<char> myStack;
    PairedParanthesis(string toScan_){
        toScan = toScan_;
    }
    bool check(){
        string Tape = toScan;
        while (!Tape.empty()){
            if (!myStack.empty()) cout << "Current Stack Top = " <<
myStack.top() <<endl;
            char cchar = char(Tape[0]);
            Tape = Tape.substr(1);

            if (cchar == '(' || cchar == '{' || cchar == '['){
                myStack.push(cchar);
            }
            else if (cchar == ')'){
                if (!myStack.empty() && myStack.top()=='(') {
                    myStack.pop();
                    continue;
                }
                else return 1;
            }
            else if (cchar == '}'){
                if (!myStack.empty() && myStack.top()=='{') {
                    myStack.pop();
                    continue;
                }
                else return 1;
            }
            else if (cchar == ']'){
                if (!myStack.empty() && myStack.top()=='[') {
                    myStack.pop();
                    continue;
                }
                else return 1;
            }
        }
        if (myStack.empty()) return 0;
    }
};
```

```
        else return 1;
    }
};

int main(){
    cout << "Input yout required infix string." << endl;
    string inpString;
    cin >> inpString;
    PairedParanthesis inf = PairedParanthesis(inpString);
    cout << "The input infix string was: " << inf.toScan << endl;
    int check = inf.check();
    if (check == 1){
        cout << "Test failed!" << endl;
    }
    else cout << "Test Passed!" << endl;
}
```

Output

```
pawanadhikari@Pawans-MacBook-Air 01_Stack % ./04_PairedParanthesis
Input yout required infix string.
(
The input infix string was: (
Test failed!
pawanadhikari@Pawans-MacBook-Air 01_Stack % ./04_PairedParanthesis
Input yout required infix string.
()
The input infix string was: ()
Current Stack Top = (
Test Passed!
pawanadhikari@Pawans-MacBook-Air 01_Stack % ./04_PairedParanthesis
Input yout required infix string.
}
The input infix string was: }
Test failed!
```